Informatika pro moderní fyziky (5) zpracování výstupních dat, procvičení OOP, složitější výstupy

František HAVLŮJ

e-mail: haf@ujv.cz

ÚJV Řež
oddělení Reaktorové fyziky a podpory palivového cyklu

akademický rok 2026/2027, 22. října 2026

1. Co jsme se naučili minule
2. Zpracování výsledků
3. Vyhodnocení
4. Načítání složitějšího výstupu

Obsah

1 Co jsme se naučili minule

2 Zpracování výsledků

3 Vyhodnocení

4 Načítání složitějšího výstupu

- použití RubyGems+Bundler
- vytvoření XLSX dokumentu
- čtení jednoduchého výstupu
- základy OOP

Kde jsme skončili

- máme třídu MCNPOutput
- v konstruktoru načte keff a odchylku

Obsah

1 Co jsme se naučili minule

2 Zpracování výsledků

3 Vyhodnocení

4 Načítání složitějšího výstupu

Skončili jsme u takovéto tabulky

něco jak toto:

```
0.0   0.0  0.94800
0.0 640.0  0.99800
0.0  64.0  0.94850
0.0 128.0  0.95000
0.0 192.0  0.95250
0.0 256.0  0.95600
...
```

– vylepšete to tedy ještě o přenásobení rozsahu

A protože přehlednost je nade vše

rádi bychom měli něco jak toto:

keff	0.0	64.0	128.0	...
0.0	0.94800	0.94900	0.95000	...
64.0	0.94850	0.94950	0.95050	...
128.0	0.95000	0.95100	0.95200	...
192.0	0.95250	0.95350	0.95450	...
256.0	0.95600	0.95700	0.95800	...
320.0	0.96050	0.96150	0.96250	...
384.0	0.96600	0.96700	0.96800	...
448.0	0.97250	0.97350	0.97450	...
512.0	0.98000	0.98100	0.98200	...
576.0	0.98850	0.98950	0.99050	...
640.0	0.99800	0.99900	1.00000	...

– ale jak nejlépe uložit data do 2D struktury?

1D a 2D pole

První možnost je mít normálně 1D pole a spočítat si index:

```
a[i + j * 11]
```

1D a 2D pole

První možnost je mít normálně 1D pole a spočítat si index:
`a[i + j * 11]`
Druhá možnost je ‘2D pole’ – ve skutečnosti v Ruby nic takového není, ale můžete mít pochopitelne pole polí (tj. každý prvek pole může být klidně pole, proč ne): `a[i][j]`

1D a 2D pole

První možnost je mít normálně 1D pole a spočítat si index:
`a[i + j * 11]`
Druhá možnost je ‘2D pole’ – ve skutečnosti v Ruby nic takového není, ale můžete mít pochopitelne pole polí (tj. každý prvek pole může být klidně pole, proč ne): `a[i][j]`
Problém je, že data nemám seřazená, takže moc nemáme dobrý způsob, jak je postupně do matice nastrkat.
Jedna možnost je nejdřív si to pole vyrobit (prvky budou nil nebo 0 nebo tak něco) a pak do něj teprv zapisovat. Druhá možnost je zapomenout na pole a udělat si to jednodušší.

Hash to the rescue

Možná se bude velmi hodit hash!

```
data = {}
data[3] = 0.99
```

Hash to the rescue

Možná se bude velmi hodit hash!

```
data = {}
data[3] = 0.99
```

no a dokonce, protože klíč může být cokoliv:

```
data = {}
data[[1,2]] = 0.99
data[[7,8]] = 1.05
puts data[[1,2]]
```

Takže nic nebrání tomu si naskládat data do hashe (= jakobymatice s volnou strukturou) a vypsat krásnou tabulku.

Chtěli bychom:

- vypsat rozumnou tabulku výsledků na terminál
- vypsat tutéž tabulku do excelu
- vykreslit grafy (x=R2, y=keff) pro každou polohu R1

Jak na to:

- vytvoře novou třídu ZKE
- jako konstruktor jí můžete dát třeba cestu k výpočtům
- jednotlivé výstupy implementujte jako metody na této třídě

```
zke = ZKE.new("../../04/data/mcnp/many_outputs"
zke.print
zke.output_xlsx("output/zke.xlsx")
zke.plot("output/zke_plots")
```

Obsah

1 Co jsme se naučili minule

2 Zpracování výsledků

3 Vyhodnocení

4 Načítání složitějšího výstupu

Chtěli bychom dále:

- pro každou polohu R1 najít kritickou polohu R2
- opět lze s úspěchem dodělat na třídu ZKE
- vypište na terminál
- vykreslete závislost do grafu (x=R1, y=R2-kritická)

Obsah

1 Co jsme se naučili minule

2 Zpracování výsledků

3 Vyhodnocení

4 Načítání složitějšího výstupu

HELIOS

Tabulka výstupů:

```
List name         : list
List Title(s)  1) This is a table
               2) of some data
               3) in many columns
               4) and has a long title!
```

	bup	kinf	ab	ab	u235	
0001	0.00E+00	1.16949	9.7053E-03	7.6469E-02	1.8806E-04	7.
0002	0.00E+00	1.13213	9.7478E-03	7.9058E-02	1.8806E-04	7.
0003	1.00E+01	1.13149	9.7488E-03	7.9070E-02	1.8797E-04	7.
0004	5.00E+01	1.13004	9.7521E-03	7.9093E-02	1.8760E-04	7.
0005	1.00E+02	1.12826	9.7559E-03	7.9218E-02	1.8714E-04	7.
0006	1.50E+02	1.12664	9.7594E-03	7.9407E-02	1.8668E-04	7.
0007	2.50E+02	1.12399	9.7657E-03	7.9869E-02	1.8577E-04	7.
0008	5.00E+02	1.12007	9.7812E-03	8.1065E-02	1.8351E-04	7.
0009	1.00E+03	1.11561	9.8203E-03	8.3169E-02	1.7914E-04	7.
0010	2.00E+03	1.10542	9.9329E-03	8.6731E-02	1.7088E-04	7.
0011	3.00E+03	1.09354	1.0067E-02	8.9717E-02	1.6316E-04	7.
0012	4.00E+03	1.08126	1.0207E-02	9.2299E-02	1.5591E-04	7.
0013	6.00E+03	1.05755	1.0474E-02	9.6562E-02	1.4258E-04	7.

Co bychom chtěli

- mít načtené jednotlivé tabulky (zatím jen jednu, ale bude jich víc)
- asi po jednotlivých sloupcích, sloupec = pole (hodnot po řádcích)
- sloupce se nějak jmenují, tedy použijeme `Hash`
- `table["kinf"]`
- pozor na `ab`, asi budeme muset vyrobit něco jako `ab1`, `ab2` (ale to až za chvíli)

Nástrahy, chytáky a podobně

- tabulka skládající se z více bloků
- více tabulek
- tabulky mají jméno `list name` a popisek `list title(s)`

Jak uspořádat data?

- pole s tabulkami + pole s názvy + pole s titulky?

Jak uspořádat data?

- pole s tabulkami + pole s názvy + pole s titulky?
- co hashe tabulky[název] a titulky[název]?

Jak uspořádat data?

- pole s tabulkami + pole s názvy + pole s titulky?
- co hashe tabulky[název] a titulky[název]?
- nejchytřeji: `{"a"=>{:title => "Table title", :data => {"kinf"=>...}}}`

Jak uspořádat data?

- pole s tabulkami + pole s názvy + pole s titulky?
- co hashe tabulky[název] a titulky[název]?
- nejchytřeji: `{"a"=>{:title => "Table title", :data => {"kinf"=>...}}}`
- “nová” syntaxe: `{"a"=>{title: "Table title", data: {"kinf"=>...}}}`

Z příkazové řádky

- a co takhle z toho udělat skript, který lze pustit s argumentem = univerzální
- `ruby read_helios.rb helios1.out`
- vypíše seznam všech tabulek, seznam jejich sloupců, počet řádků
- pole `ARGV` – seznam všech argumentů
- vylepšení – provede pro všechny zadané soubory: `ruby read_helios.rb helios1.out helios2.out` (tip: využívejte vlastní metody, kde to jen jde)

A to je vše, přátelé!

